

# React Router — Cheat Sheet

---

React Router è una libreria che permette di creare un'app React con più pagine, ognuna associata a un URL diverso.

Questa guida usa la **modalità dichiarativa** di React Router e un piccolo e-commerce come esempio.

## 1. Da una pagina React a una SPA con più pagine

Una normale applicazione React mostra inizialmente un solo componente principale, per esempio `<App />`.

In un sito, però, vogliamo poter visitare pagine diverse:

- `/` → Homepage
- `/about` → Chi siamo
- `/products` → Lista dei prodotti
- `/products/3` → Dettaglio del prodotto con id 3

React Router osserva l'URL e decide quale componente mostrare.

URL nel browser → React Router trova la rotta → React mostra il componente

L'app rimane una **SPA** (*Single Page Application*): durante la navigazione il browser non scarica una nuova pagina HTML. React sostituisce solo i componenti necessari, rendendo il passaggio tra le pagine più veloce.

Parole importanti

- **URL**: l'indirizzo mostrato nel browser, per esempio `/products/3`.
- **Rotta** (*route*): una regola che associa un URL a un componente.
- **Routing**: il sistema che sceglie quale pagina mostrare.
- **Navigazione lato client**: cambio di pagina gestito da React, senza ricaricare il documento HTML.

## 2. Installazione e BrowserRouter

Installa la versione attuale della libreria:

```
npm i react-router
```

Nel file `main.jsx`, avvolgi `<App />` con `<BrowserRouter>`:

```
import { createRoot } from 'react-dom/client';
import { BrowserRouter } from 'react-router';
import { App } from './App.jsx';
```

```
createRoot(document.getElementById('root')).render(  
  <BrowserRouter>  
    <App />  
  </BrowserRouter>,  
);
```

**BrowserRouter** collega React Router al browser. In questo modo l'app può:

- leggere l'URL corrente;
- cambiare URL senza ricaricare la pagina;
- usare la cronologia del browser, quindi i pulsanti Indietro e Avanti.

Deve contenere tutti i componenti che utilizzano le funzionalità di React Router.

### 3. Creare le pagine e configurare le rotte

In React, una “pagina” è semplicemente un componente:

```
function Home() {  
  return <h1>Benvenuto nel nostro store!</h1>;  
}  
  
function About() {  
  return <h1>Chi siamo</h1>;  
}  
  
function Products() {  
  return <h1>Prodotti</h1>;  
}
```

Nel componente **App** colleghiamo gli URL alle pagine:

```
import { Route, Routes } from 'react-router';  
  
export function App() {  
  return (  
    <Routes>  
      <Route path="/" element={<Home />} />  
      <Route path="/about" element={<About />} />  
      <Route path="/products" element={<Products />} />  
    </Routes>  
  );  
}
```

Come leggere questo codice

- **Routes** contiene tutte le rotte dell'app.

- Ogni **Route** rappresenta una regola.
- **path** è l'URL da riconoscere.
- **element** è il componente da mostrare quando l'URL corrisponde.

Per esempio:

```
<Route path='/products' element={<Products />} />
```

significa: "quando l'URL è **/products**, mostra il componente **<Products />**".

## 4. Navigare con Link

Per spostarsi tra le pagine interne usa **Link**:

```
import { Link } from 'react-router';

function Home() {
  return (
    <>
      <h1>Benvenuto nel nostro store!</h1>
      <Link to='/products'>Scopri i prodotti</Link>
    </>
  );
}
```

La prop **to** contiene l'URL da raggiungere.

Per i collegamenti interni non usare:

```
<a href='/products'>Prodotti</a>
```

Un normale tag **<a>** richiede al browser una nuova pagina e ricarica l'app. **Link**, invece, cambia l'URL usando React Router e mantiene la navigazione lato client.

## 5. Navbar e NavLink

Nella Navbar conviene usare **NavLink**. Funziona come **Link**, ma permette anche di capire se il collegamento corrisponde alla pagina corrente.

```
import { NavLink } from 'react-router';

function Navbar() {
  return (
    <nav>
      <NavLink to='/' end>
```

```
      Home
    </NavLink>

    <NavLink to='/about'>Chi siamo</NavLink>
    <NavLink to='/products'>Prodotti</NavLink>
  </nav>
);
}
```

Quando un **NavLink** è attivo, React Router aggiunge automaticamente la classe CSS **active**:

```
a.active {
  color: tomato;
  font-weight: bold;
}
```

La prop **end** sul link Home indica che deve essere attivo solo quando l'URL è esattamente **/**. Senza **end**, **/** risulterebbe l'inizio anche di **/about** e **/products**.

In breve:

- usa **Link** per i normali collegamenti;
- usa **NavLink** quando serve evidenziare la pagina attiva.

## 6. Layout e Outlet

Navbar e Footer devono rimanere visibili in tutte le pagine. Invece di ripeterli in ogni componente, possiamo creare un **Layout condiviso**.

```
import { Outlet } from 'react-router';

function Layout() {
  return (
    <>
      <Navbar />

      <main>
        <Outlet />
      </main>

      <Footer />
    </>
  );
}
```

**Outlet** è uno spazio vuoto che React Router sostituisce con la pagina corrispondente all'URL:

|           |                          |
|-----------|--------------------------|
| Layout    |                          |
| └─ Navbar | sempre visibile          |
| └─ Outlet | Home, About, Products... |
| └─ Footer | sempre visibile          |

Per usare il Layout, le pagine diventano rotte figlie:

```
export function App() {
  return (
    <Routes>
      <Route element={<Layout />}>
        <Route index element={<Home />} />
        <Route path='about' element={<About />} />
        <Route path='products' element={<Products />} />
      </Route>
    </Routes>
  );
}
```

La rotta del Layout non ha un **path**: non aggiunge parti all'URL, ma avvolge le rotte figlie.

La rotta con **index** è la pagina predefinita del Layout. In questo caso mostra **<Home />** quando l'URL è **/**.

## 7. Rotte dinamiche

Per il dettaglio di ogni prodotto potremmo creare rotte separate:

```
/products/1
/products/2
/products/3
```

I prodotti possono però essere molti. Non vogliamo scrivere una **Route** diversa per ogni id.

Creiamo quindi una **rotta dinamica**:

```
<Route path='products/:id' element={<ProductDetail />} />
```

I due punti indicano che **:id** è un valore variabile:

- **/products/1** → **id** vale "1"
- **/products/7** → **id** vale "7"
- **/products/15** → **id** vale "15"

Il nome **id** è scelto da noi. Potremmo chiamarlo **productId**, ma dovremmo poi usare lo stesso nome anche in **useParams**.

## Creare il link per ogni prodotto

Nella pagina del listato costruiamo l'URL usando l'id ricevuto dall'API:

```
import { Link } from 'react-router';

function Products({ products }) {
  return (
    <section>
      <h1>Prodotti</h1>

      {products.map((product) => (
        <article key={product.id}>
          <h2>{product.title}</h2>
          <Link to={` /products/${product.id}`}>Vedi dettagli</Link>
        </article>
      ))}
    </section>
  );
}
```

Se `product.id` vale 4, il template literal crea l'URL `/products/4`.

## 8. Leggere l'id con useParams

`useParams` è un hook che restituisce i valori dinamici presenti nell'URL.

```
import { useParams } from 'react-router';

function ProductDetail() {
  const { id } = useParams();

  return <p>Stai visualizzando il prodotto {id}</p>;
}
```

Il procedimento completo è:

```
Link a /products/4
  ↓
la rotta products/:id riconosce l'URL
  ↓
useParams() restituisce { id: "4" }
```

I parametri dell'URL sono sempre **stringhe**. Se devi fare un calcolo, convertili:

```
const productId = Number(id);
```

Il nome deve corrispondere a quello scritto nella rotta:

```
// Rotta
<Route path='products/:productId' element={<ProductDetail />} />;

// Componente
const { productId } = useParams();
```

## 9. Caricare il prodotto corretto

Possiamo usare l'id per chiamare l'endpoint di dettaglio della Fake Store API:

```
import { useEffect, useState } from 'react';
import { useParams } from 'react-router';

function ProductDetail() {
  const { id } = useParams();
  const [product, setProduct] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    setLoading(true);

    fetch(`https://fakestoreapi.com/products/${id}`)
      .then((response) => response.json())
      .then((data) => {
        setProduct(data);
        setLoading(false);
      });
  }, [id]);

  if (loading) {
    return <p>Caricamento...</p>;
  }

  return (
    <article>
      <h1>{product.title}</h1>
      <img src={product.image} alt={product.title} />
      <p>{product.description}</p>
      <p>{product.price} €</p>
    </article>
  );
}
```

Quando cambia l'id nell'URL, cambia la dipendenza `[id]: useEffect` esegue una nuova richiesta e carica il nuovo prodotto.

Il loading è gestito con `useState`: è utile durante la richiesta, ma non è una funzionalità di React Router.

## 10. Navigazione programmatica con `useNavigate`

`Link` e `NavLink` si usano quando l'utente vede e clicca un collegamento.

In alcuni casi, invece, è il nostro codice a decidere quando cambiare pagina. Per questo esiste `useNavigate`.

```
import { useNavigate } from 'react-router';

function Example() {
  const navigate = useNavigate();

  function goToProducts() {
    navigate('/products');
  }

  return <button onClick={goToProducts}>Torna ai prodotti</button>;
}
```

Altri esempi:

```
navigate('/products'); // Va a un URL preciso
navigate(-1); // Torna indietro nella cronologia
navigate(1); // Va avanti nella cronologia
```

### Prodotto inesistente

Se l'API non restituisce un prodotto valido, possiamo riportare l'utente al listato:

```
const navigate = useNavigate();

fetch(`https://fakestoreapi.com/products/${id}`)
  .then((response) => response.json())
  .then((data) => {
    if (!data || !data.id) {
      navigate('/products');
      return;
    }

    setProduct(data);
  });
```

## Prodotto precedente e successivo

Partendo dall'id corrente possiamo calcolare l'URL del prodotto vicino:

```
const { id } = useParams();
const navigate = useNavigate();
const productId = Number(id);

<button
  onClick={() => navigate(`/products/${productId - 1}`)}
  disabled={productId <= 1}
>
  Precedente
</button>;

<button onClick={() => navigate(`/products/${productId + 1}`)}>
  Successivo
</button>;
```

`navigate()` cambia l'URL. La rotta dinamica rimane la stessa, ma `useParams` legge il nuovo id e il componente carica il nuovo prodotto.

## 11. Pagina 404

La pagina 404 viene mostrata quando nessuna delle altre rotte corrisponde all'URL:

```
function NotFound() {
  return (
    <>
      <h1>404 - Pagina non trovata</h1>
      <Link to="/">Torna alla Homepage</Link>
    </>
  );
}
```

Inserisci per ultima una rotta con `path='*'`:

```
<Route path='*' element={<NotFound />} />
```

L'asterisco significa "qualsiasi altro percorso".

## Configurazione finale delle rotte

```
<Routes>
  <Route element={<Layout />}>
```

```
<Route index element={<Home />} />
<Route path='about' element={<About />} />
<Route path='products' element={<Products />} />
<Route path='products/:id' element={<ProductDetail />} />
<Route path='*' element={<NotFound />} />
</Route>
</Routes>
```

## Riepilogo

| Strumento            | A cosa serve                                  |
|----------------------|-----------------------------------------------|
| <b>BrowserRouter</b> | Attiva React Router nell'app                  |
| <b>Routes</b>        | Contiene l'elenco delle rotte                 |
| <b>Route</b>         | Collega un URL a un componente                |
| <b>Link</b>          | Crea un collegamento interno                  |
| <b>NavLink</b>       | Crea un link che può mostrare lo stato attivo |
| <b>Outlet</b>        | Mostra la pagina figlia dentro un Layout      |
| <b>useParams()</b>   | Legge i segmenti dinamici dell'URL            |
| <b>useNavigate()</b> | Cambia pagina tramite codice                  |
| <b>path='*'</b>      | Intercetta gli URL non riconosciuti           |

## Errori comuni

- Dimenticare **BrowserRouter** attorno ad **<App />**.
- Usare **<a href>** invece di **Link** per una pagina interna.
- Creare un Layout senza inserire **Outlet**.
- Scrivere **:id** nella rotta e leggere un nome diverso con **useParams**.
- Dimenticare che l'id letto dall'URL è una stringa.
- Usare **useNavigate** per ogni link: per la navigazione normale sono preferibili **Link** e **NavLink**.

## Documentazione ufficiale

- [Installazione](#)
- [Routing](#)
- [Navigazione](#)
- [Valori della URL](#)